

Automated E-Learning Data Pipeline

Technical Documentation

System Architecture, Database Schema & Process Diagrams — v1.0

1. Solution Overview	3
2. System Architecture	3
2.1 Local Architecture	3
2.2 Cloud Architecture (Microsoft Fabric)	3
2.3 Technology Stack	3
3. Database Schema (Star Schema)	4
3.1 Dimension Tables	4
3.2 Fact Table — fact_offerings	4
3.3 Referential Integrity & Testing	5
4. dbt Transformation Layers	5
4.1 Staging	5
4.2 Intermediate	5
4.3 Snapshots (SCD Type 2)	5
4.4 Marts	5
4.5 Macros	6
5. Orchestration Details	6
5.1 Local — Airflow DAG: course_data_pipeline	6
5.2 Cloud — Fabric Pipeline: "Pipelining"	6
6. Diagrams	6
6.1 Data Flow Diagrams (DFD)	6
6.1.1 Context-Level DFD — Local Solution	6
6.1.2 Level-1 (Detailed) DFD — Local Solution	7
6.1.3 Context-Level DFD — Cloud Solution	7
6.1.4 Level-1 (Detailed) DFD — Cloud Solution	7
6.2 Sequence Diagrams	8
6.2.1 Sequence Diagram — Local Pipeline Run	8
6.2.2 Sequence Diagram — Cloud Pipeline Run	8
6.3 Activity Diagrams	9
6.3.1 Activity Diagram — Local Workflow	9
6.3.2 Activity Diagram — Cloud Workflow	9
7. Non-Functional Notes	10
8. Support & Ownership	10

1. Solution Overview

The project ingests course catalog data from three e-learning platforms — Coursera, Udemy, and Udacity — and unifies it into a single, analytics-ready Kimball-style Star Schema. Two parallel implementations of the same logical pipeline exist:

- Local Solution: Apache Airflow (Docker Compose) orchestrating dbt-duckdb against a local DuckDB file.
- Cloud Solution: Microsoft Fabric pipeline orchestrating a Fabric "Data Build Tool" (dbt) job against a Fabric Lakehouse/Warehouse, refreshing a Power BI semantic model and dashboard.

Both share the same dbt project structure: staging → intermediate → snapshots (SCD Type 2) → marts (8 dimensions + 1 fact), plus 73+ automated dbt tests.

2. System Architecture

2.1 Local Architecture

- Extraction: standalone Python scrapers (coursera_scraper, udacity_scraper, udemy_scraper) produce *_final_data.csv files.
- Orchestration: Apache Airflow 2.9.3 (LocalExecutor) running in Docker Compose alongside a Postgres metadata database.
- Transformation: dbt-duckdb executes the dbt project in 5 ordered tasks (deps → seed → snapshot → run → test) to avoid race conditions between snapshot and model builds.
- Storage: a single DuckDB file (dbt_local.duckdb) holding raw, staging, intermediate, snapshot, and mart tables — mounted into the Airflow containers as a volume.
- Consumption: Data Engineers query DuckDB directly; any DuckDB-compatible BI client can connect for analysis.

2.2 Cloud Architecture (Microsoft Fabric)

- Extraction/Ingestion: a Fabric Notebook (Notebook1) loads/refreshes raw course data into the Fabric Lakehouse/Warehouse (schema dbo).
- Orchestration: a Fabric Pipeline ("Pipelining") chains three activities — Notebook1 → dbt job1 (InvokeDataBuildToolJob) → Semantic model refresh1 (PBISemanticModelRefresh) — each gated on the previous activity's Succeeded status.
- Transformation: the same dbt project runs as a managed Fabric "Data Build Tool" job (operation: build, 4 threads) against the Fabric Warehouse.
- Storage: Fabric Lakehouse/Warehouse tables (raw_data.dbo schema for sources; mart tables for the star schema).
- Consumption: a Power BI semantic model (PowerBI_Dashboard.pbix) is refreshed transactionally across all 9 mart tables, then explored via Power BI reports by Analysts.

2.3 Technology Stack

Layer	Local Solution	Cloud Solution
Orchestration	Apache Airflow 2.9.3 (Docker Compose, LocalExecutor)	Microsoft Fabric Pipeline (activity chaining)
Ingestion	Python scrapers (requests/curl_cffi, BeautifulSoup-style parsers)	Fabric Notebook (TridentNotebook)
Transformation	dbt-core + dbt-duckdb	dbt via Fabric Data Build Tool job
Database	DuckDB (single-file OLAP)	Fabric Lakehouse / Warehouse
Metadata store	PostgreSQL 13 (Airflow metadata only)	Managed by Fabric
BI / Visualization	Local SQL clients / BI tools	Power BI (semantic model + report)
Containerization	Docker, Docker Compose	N/A (managed PaaS)

3. Database Schema (Star Schema)

Both environments build an identical Kimball-style dimensional model: 8 dimension tables surrounding 1 central fact table, `fact_offerings`.

3.1 Dimension Tables

Table	Primary Key(s)	Key Attributes
<code>dim_date</code>	<code>date_id</code> (PK)	<code>calendar_date</code> , <code>year</code> , <code>quarter</code> , <code>month</code> , <code>month_name</code> , <code>day</code> , <code>day_of_week</code> , <code>is_weekend</code>
<code>dim_domain</code>	<code>domain_id</code> (PK)	<code>domain_name</code> (e.g. Data Science, Business, IT)
<code>dim_instructor</code>	<code>instructor_id</code> (PK)	<code>instructor_name</code> (standardized, 'Unknown Instructor' default)
<code>dim_language</code>	<code>language_id</code> (PK)	<code>language_name</code> (normalized language strings)
<code>dim_level</code>	<code>level_id</code> (PK)	<code>level_name</code> (standardized difficulty: Beginner/Intermediate/Advanced)
<code>dim_offering_type</code>	<code>offering_type_id</code> (PK)	<code>offering_type</code> (Course, Specialization, Nanodegree, etc.)
<code>dim_platform</code>	<code>platform_id</code> (PK)	<code>platform_name</code> (Coursera, Udemy, Udacity)
<code>dim_offering</code>	<code>offering_sk</code> (PK), <code>offering_id</code>	<code>title</code> , <code>description</code> , <code>skills</code> , <code>course_url</code> , <code>effective_from</code> , <code>effective_to</code> , <code>is_current</code> (SCD Type 2, built from <code>offering_snapshot</code>)

3.2 Fact Table — `fact_offerings`

Grain: one row per current course offering (one per `offering_sk` from `dim_offering` where `is_current = true`).

Column	Type / Role	Description
<code>offering_sk</code>	FK → <code>dim_offering.offering_sk</code>	Surrogate key of the current course version
<code>platform_id</code>	FK → <code>dim_platform.platform_id</code>	Source platform
<code>domain_id</code>	FK → <code>dim_domain.domain_id</code>	Subject domain
<code>level_id</code>	FK → <code>dim_level.level_id</code>	Difficulty level
<code>language_id</code>	FK → <code>dim_language.language_id</code>	Course language

Column	Type / Role	Description
instructor_id	FK → dim_instructor.instructor_id	Instructor (-1 if unknown)
offering_type_id	FK → dim_offering_type.offering_type_id	Course / Specialization / Nanodegree, etc. (-1 if unknown)
date_id	FK → dim_date.date_id	Last-updated date (-1 if unmatched)
price, rating, review_count	Measures	Raw numeric metrics
enrolled_students, duration_hours	Measures	Raw numeric metrics
popularity_score, engagement_rate_pct	Calculated measures	Derived in int_courses_standardized
weighted_rating, cost_per_hour, estimated_total_cost, value_score	Calculated measures	Derived analytical metrics for the recommendation engine

3.3 Referential Integrity & Testing

dbt enforces not_null and unique constraints on every dimension's primary key, and relationships tests on every foreign key in fact_offerings against its parent dimension, as part of 73+ automated tests run via dbt test (local) or embedded in the dbt build/test step (cloud).

4. dbt Transformation Layers

4.1 Staging

stg_coursera, stg_udemy, stg_udacity — extract raw source tables (coursera_final_data, udemy_final_data, udacity_final_data), rename columns to a common standard, cast types, and apply base filtering. Materialized as views.

4.2 Intermediate

- int_courses_combined — unions all three staging models into a single master table.
- int_courses_standardized — standardizes categorical text (duration buckets, language normalization) and computes derived metrics (popularity_score, value_score, etc.) used downstream by fact_offerings.

4.3 Snapshots (SCD Type 2)

offering_snapshot tracks changes to title, description, skills, and url on each course. When any tracked column changes, dbt closes the old record (dbt_valid_to set) and inserts a new one (dbt_valid_from set), preserving full history for the recommendation engine.

4.4 Marts

8 dimension models plus fact_offerings, all materialized as tables for query performance. dim_offering is built directly from offering_snapshot, deriving a row-numbered surrogate key (offering_sk) and an is_current flag.

4.5 Macros

- `parse_duration.sql` — normalizes free-text duration strings (e.g. "300 mins" vs "5 hours") into a standard unit.
- `standardize_language.sql` — normalizes inconsistent language naming across platforms.

5. Orchestration Details

5.1 Local — Airflow DAG: `course_data_pipeline`

Linear task chain (chosen over a single "dbt build" to avoid a dependency-cycle race condition between `offering_snapshot` and `int_courses_standardized` on a fresh database):

```
dbt_deps >> dbt_seed >> dbt_snapshot >> dbt_run >> dbt_test
```

Default args: owner `course_data_team`, retries=1 (2-minute delay), schedule_interval=None (manual trigger only), catchup=False.

5.2 Cloud — Fabric Pipeline: "Pipelining"

Three chained activities, each depending on the previous activity's Succeeded status:

```
Notebook1 → dbt job1 (InvokeDataBuildToolJob, build, threads=4) → Semantic model refresh1 (PBISemanticModelRefresh, transactional commit)
```

The semantic model refresh targets 9 tables: `dim_date`, `dim_instructor`, `dim_language`, `dim_level`, `dim_offering_type`, `dim_platform`, `dim_offering`, `dim_domain`, and `fact_offerings`.

6. Diagrams

6.1 Data Flow Diagrams (DFD)

6.1.1 Context-Level DFD — Local Solution

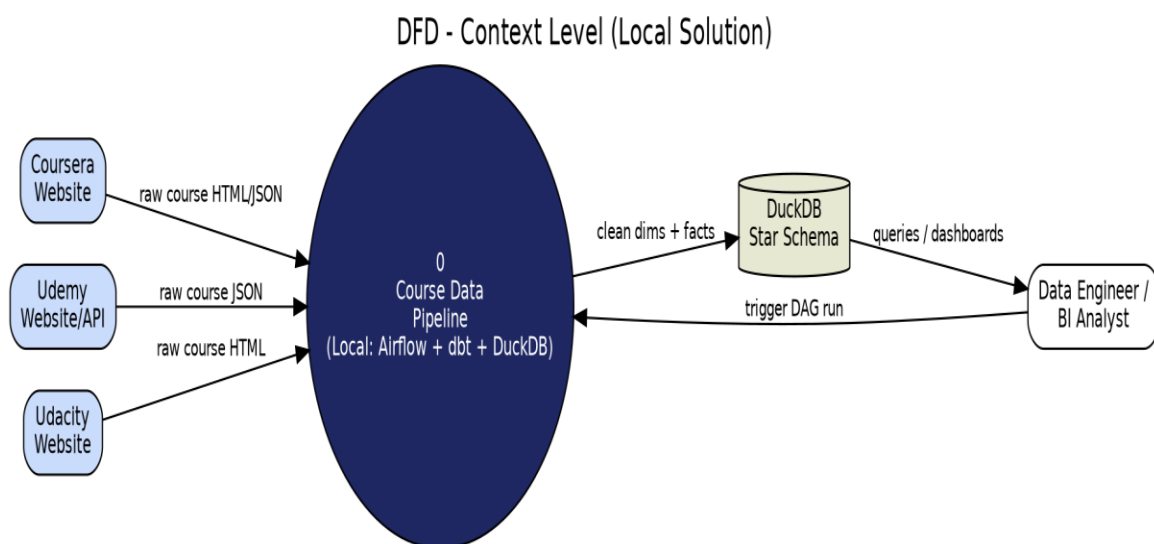


Figure 1. Context-level DFD, Local Solution.

6.1.2 Level-1 (Detailed) DFD — Local Solution

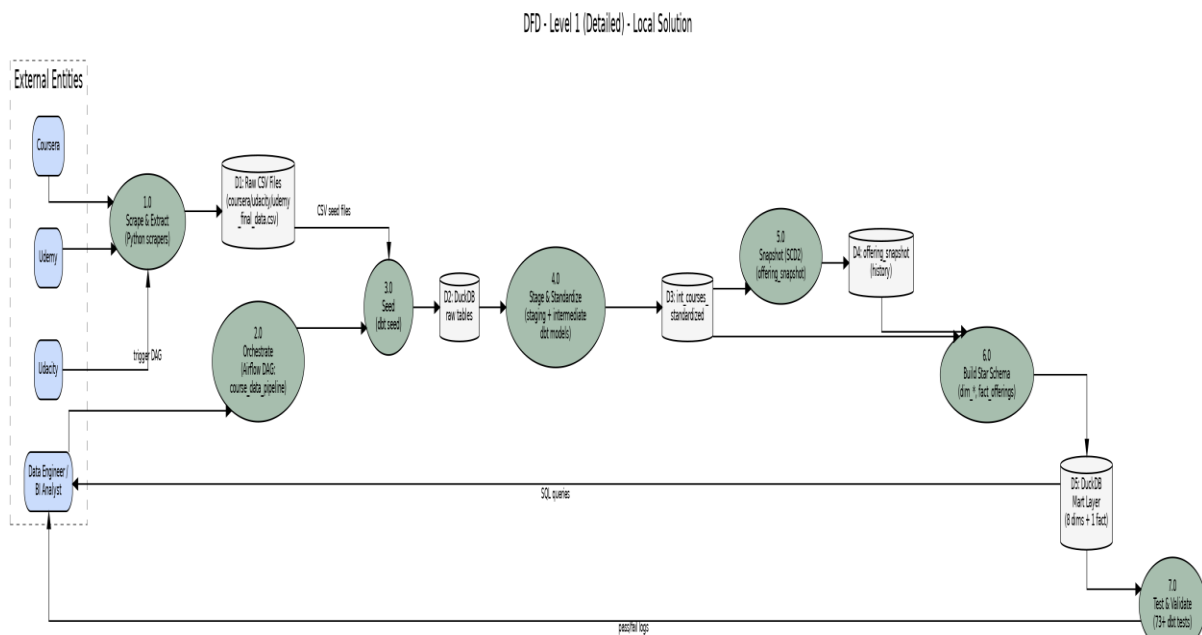


Figure 2. Detailed DFD, Local Solution (extraction → seed → standardize → snapshot → marts → test).

6.1.3 Context-Level DFD — Cloud Solution

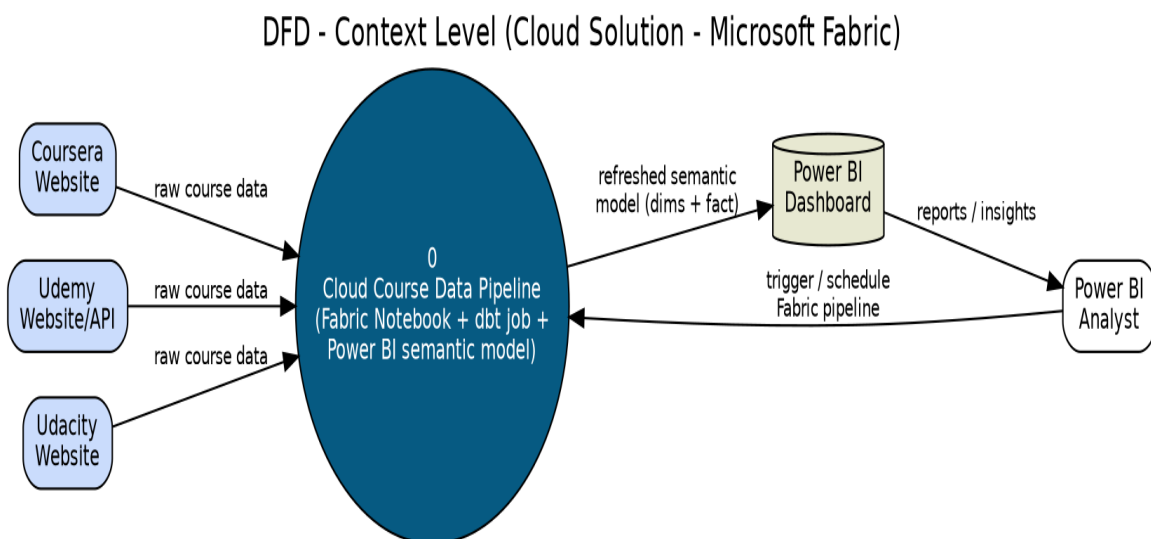


Figure 3. Context-level DFD, Cloud Solution (Microsoft Fabric).

6.1.4 Level-1 (Detailed) DFD — Cloud Solution

DFD - Level 1 (Detailed) - Cloud Solution (Microsoft Fabric)

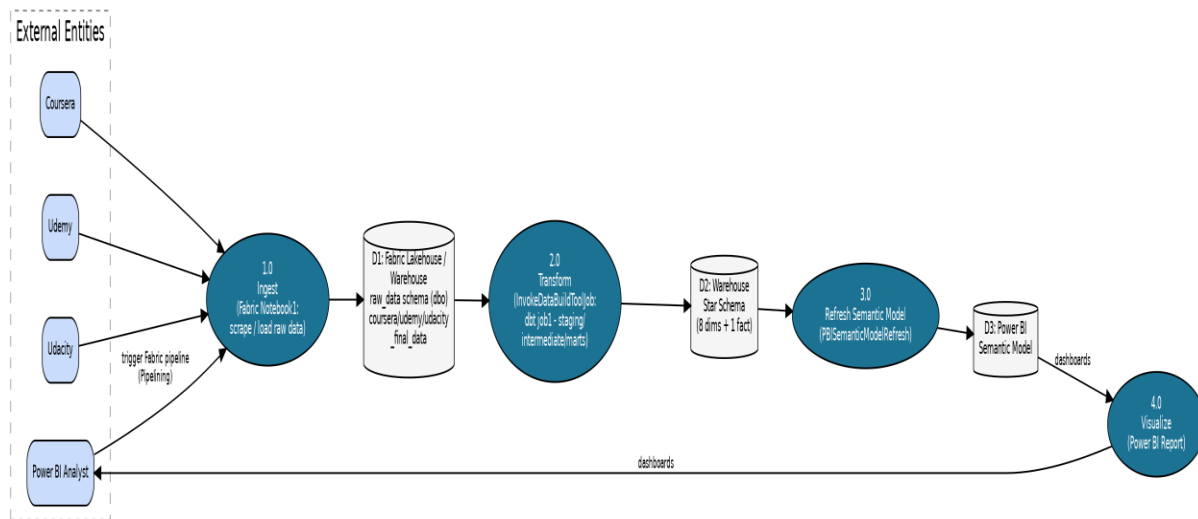


Figure 4. Detailed DFD, Cloud Solution (ingest → transform → semantic refresh → visualize).

6.2 Sequence Diagrams

6.2.1 Sequence Diagram — Local Pipeline Run

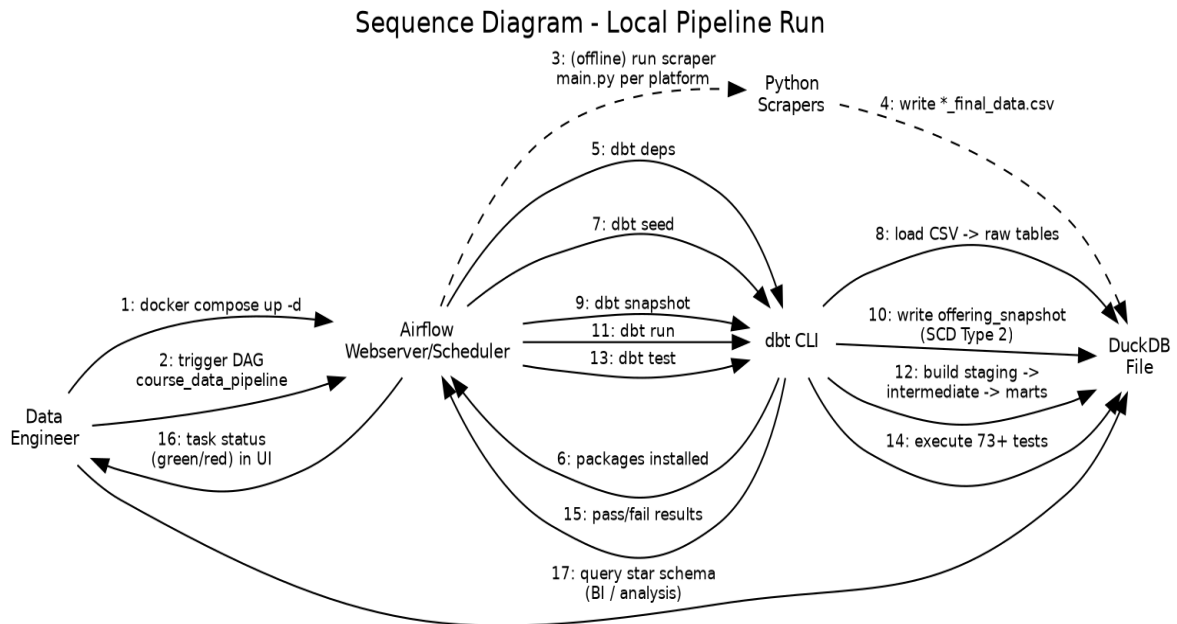


Figure 5. Interaction sequence for a manual local pipeline run.

6.2.2 Sequence Diagram — Cloud Pipeline Run

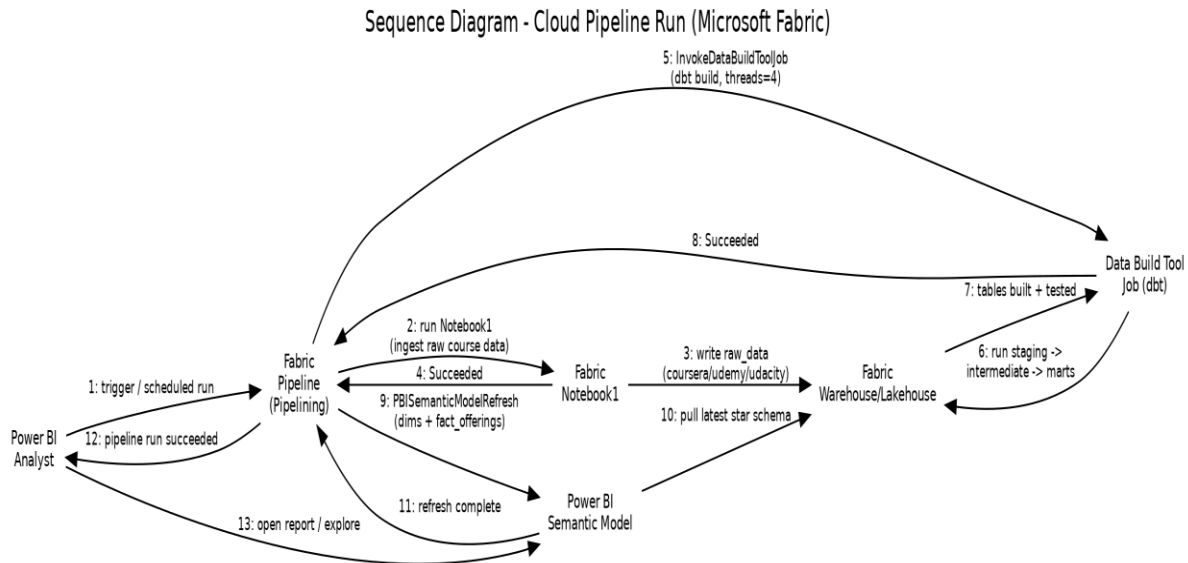


Figure 6. Interaction sequence for a Fabric pipeline run.

6.3 Activity Diagrams

6.3.1 Activity Diagram — Local Workflow

Activity Diagram - Local Pipeline Workflow

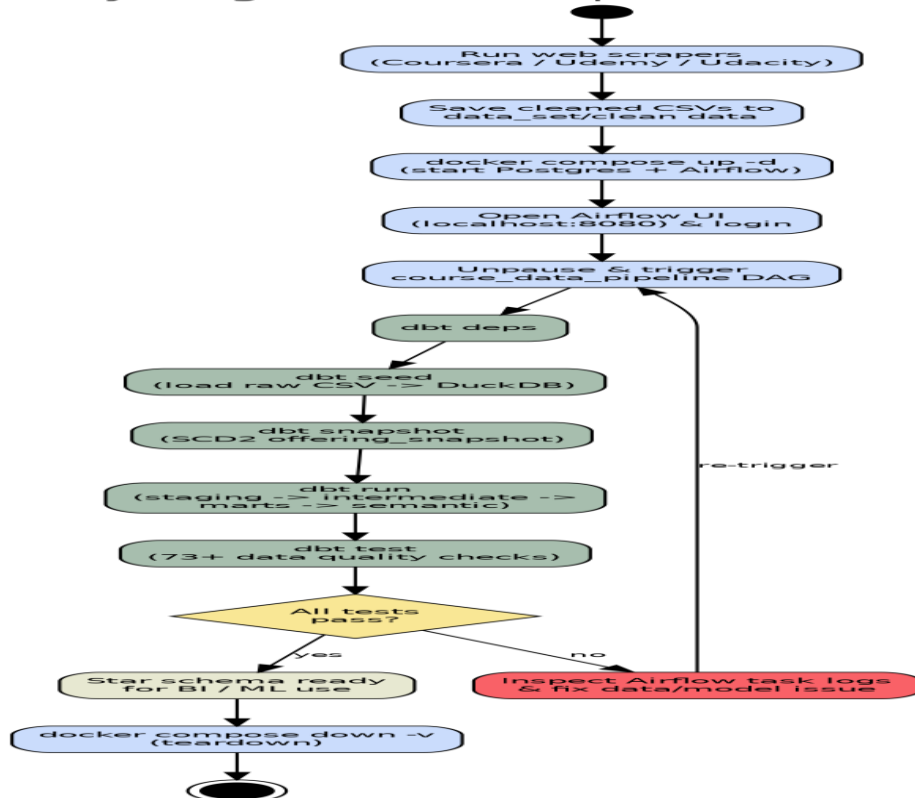


Figure 7. End-to-end local workflow, including the test pass/fail decision point.

6.3.2 Activity Diagram — Cloud Workflow

Activity Diagram - Cloud Pipeline Workflow (Microsoft Fabric)

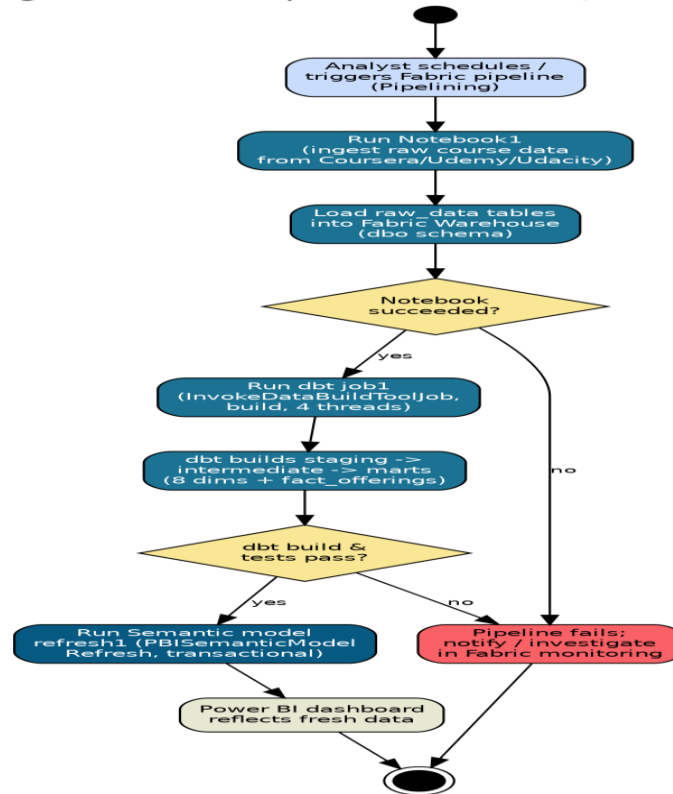


Figure 8. End-to-end cloud workflow, including decision points for Notebook and dbt success.

7. Non-Functional Notes

- Data Quality: 73+ dbt tests covering not_null, unique, accepted_values, and relationships, executed on every run in both environments.
- Historical Accuracy: SCD Type 2 snapshotting prevents algorithm decay in the downstream recommendation engine when course content changes.
- Idempotency: dbt seed/run/snapshot/test steps are designed to be safely re-run; the Airflow DAG also defines 1 retry with a 2-minute back-off per task.
- Portability: the dbt project (models/macros/snapshots) is shared in structure between Local and Cloud, differing mainly in the source schema (main vs dbo) and database adapter.

8. Support & Ownership

Project repository: Mahmoud-E-28/Automated_E-Learning_Data_Pipeline. For pipeline failures, escalate with the failing task/activity name and relevant log excerpt to the data engineering owner of the project.